

Editor Tools

This page describes the set of tools that assist MIDI development within the Unity editor.

Tool	Mode	Menu
MIDI Monitor	Play only	Window > MIDI > Monitor
Virtual MIDI Controller	Play only	Window > MIDI > Virtual Controller
SMF Preview	Edit only	Window > MIDI > SMF Preview
Project Settings	Edit	Edit > Project Settings > MIDI

MIDI Monitor

An editor window that displays MIDI input/output messages in a real-time list during Play mode. It provides functionality equivalent to a DAW's MIDI monitor, improving debugging efficiency during development.

Note: This tool is **Play mode only**. In Edit mode (non-Play), no MIDI messages are displayed.

How to open

Window > MIDI > Monitor

Displayed content

Column	Content
Time	Relative time since Play started (<code>mm:ss.fff</code>)
Dir	<code>IN</code> (input) / <code>OUT</code> (output)
Device	Device ID (abbreviated; full text shown in a tooltip)
Ch	MIDI channel
Type	Message type
Detail	Note number, CC number, note name, etc.

Messages shown in the initial release

Input (IN)

- Note On / Note Off
- Control Change (CC)
- Program Change (PC)
- Device connect / disconnect events

Output (OUT)

- Note On / Note Off
- Control Change (CC)
- Program Change (PC)

For Note messages, the Detail column also shows the note name via `MidiNoteUtility` (e.g. `C4 (60)`).

Toolbar

Action	Description
Clear	Clear the log
Export	Export the filtered log to a CSV file
Auto Scroll	Automatically scroll when a new message is added
Max Lines	Maximum number of lines to retain (default 1000, max 10000)
Ch 1-16	Switch channel display between 0–15 / 1–16

Filters

Filter	Options
Direction	All / IN / OUT
Device	List of connected devices (All = all devices)
Ch	All / 0–15 (or 1–16 display)
Type	All / NoteOn / NoteOff / CC / PC / Device
Search	Partial-match search on the Detail and Device columns

Filter settings are saved to `EditorPrefs` and are retained the next time the window is opened.

How it works

Capturing input messages

When Play mode starts, a `[MIDI Monitor Proxy]` GameObject is created automatically and registered as an event handler with `MidiManager`. Received MIDI events are appended to the monitor's log buffer.

Capturing output messages

When `MidiManager`'s send methods (Note On/Off, CC, PC) execute, OUT logs are recorded via an editor-only hook (`#if UNITY_EDITOR`). Sends from `MidiSend` (the fluent API) are displayed the same way.

When Play mode ends

The Proxy GameObject is automatically destroyed and its registration with `MidiManager` is removed. The contents of the log buffer remain on the window even after Play ends (they can be cleared on the next Play).

Recommended usage

1. Open `Window > MIDI > Monitor`
2. Play a sample scene (`Assets/MIDI/Samples/Scenes/`)
3. Connect a MIDI device, or send from a script with `MidiSend.To(...).NoteOn(...)`
4. Check the IN / OUT logs

Send-test example:

```
using jp.kshoji.unity.midi.util;  
  
// Run during Play  
MidiSend.ToFirstOutput().Channel(0).NoteOn(60, 127);
```

Virtual MIDI Controller

A simulator that lets you send Note / CC / Program Change / Pitch Bend from the editor without a physical MIDI device.

How to open

```
Window > MIDI > Virtual Controller
```

Features

- 2-octave keyboard (C3–B4)
- 16 CC sliders (CC numbers saved to EditorPrefs)
- Channel (1–16) / group / velocity settings
- Output device selection (when nothing is connected, an `editor:virtual-controller` virtual output is created automatically)
- All Notes Off / Panic (CC 120 + 123)

Usage

1. Play a scene that calls `MidiManager.InitializeMidi()`, such as a sample scene
2. Open `Window > MIDI > Virtual Controller`
3. Operate the keyboard or CC sliders
4. Check the OUT messages in `Window > MIDI > Monitor`

SMF Preview / Import

Load `.mid` files in Edit mode, inspect the track and event lists, and export to a `MidiSequenceAsset`.

How to open

```
Window > MIDI > SMF Preview  
Assets > Import MIDI File...
```

Features

- Drag-and-drop loading of `.mid` files
- Track list / event list (Tick, time, type, Detail)
- Meta info display such as tempo, time signature, and track names
- Text-based note list (Note Roll)
- `MidiSequenceAsset` / JSON export

The exported `MidiSequenceAsset` can be played back with `SmfPlayer` from [SMF Tools](#).

Project Settings

Manage the MIDI Plugin's global settings from `Edit > Project Settings > MIDI`.

Settings

Section	Items
Devices	<code>defaultInputDeviceId</code> / <code>defaultOutputDeviceId</code> (empty = auto-select)
Bluetooth MIDI	<code>autoScanBleOnInit</code> , <code>bleScanTimeoutMs</code> (0 = unlimited)
RTP-MIDI	<code>rtpMidiPort</code> (default 5004), <code>rtpMidiSessionName</code>
Debug	<code>logLevel</code> (None / Error / Warning / Info / Verbose), <code>enableMidiMonitorOnPlay</code>
Development	<code>developmentBuildOnlyVerboseLog</code>

On the first import, `Assets/MIDI/Resources/MidiProjectSettings.asset` is created automatically. When `MidiManager.InitializeMidi()` completes, the BLE auto-scan and RTP-MIDI server startup settings are applied.

Future editor features (not yet implemented)

Feature	Overview
Device Browser	<code>Window > MIDI > Device Browser</code> — device list, Vendor/Product ID, test send, ID copy
Scene View debug overlay	Display MIDI state on the scene during Play
Latency measurement (RTT tool)	Editor send→receive round-trip UI (for BLE / RTP-MIDI evaluation). Runtime tap calibration is already available via Foundation <code>MidiLatencyCalibrator</code>

Source files

File	Role
MidiMonitorWindow.cs and others	MIDI Monitor
VirtualMidiControllerWindow.cs	Virtual Controller
SmfPreviewWindow.cs / SmfImportUtility.cs	SMF Preview
MidiProjectSettings.cs	Settings ScriptableObject
MidiProjectSettingsProvider.cs	Project Settings UI
MidiManager.ProjectSettings.cs	Runtime integration

Related docs

- [SMF Tools \(SmfPlayer / MidiRecorder\)](#)
- [Utilities \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [Notes on Editor and Lifecycle](#)
- [Getting Started](#)
- [Samples](#)